EXPENSE TRACKER-PROJECT REPORT

Submitted by: - Avijeet Gupta

Registration no.: - 26BAI10367

Course: - Introduction to Problem Solving and Programming

Platform: - VITyarthi - Build Your Own Project

1. Introduction: -

Keeping track of daily spending is a routine but often neglected part of personal money management. Personal Finance & Expense Tracker is a desktop application designed to give users a quick, private way to record and monitor their everyday expenses. Unlike spreadsheets or cloud-based finance apps that require sign-ups and bank access, it stores all data locally on the user's own machine, allowing students, professionals, and general users to log expenses by category and instantly see how much they have spent, without any internet connection or third-party data sharing.

2. Problem Statement: -

The Problem:

Manual expense tracking using notebooks or scattered notes often leads to forgotten entries and an inaccurate picture of spending. Most existing finance apps require account creation and access to sensitive bank details, which raises privacy concerns for users who simply want a lightweight way to log expenses.

The Solution:

A Python based client-side application that lets a user record an expense with an amount, category, and note in a few clicks, validates the entry automatically, and keeps a local, persistent history of all transactions using a simple JSON file, with no internet connection or external account required.

3. Functional Requirements: -

The system provides three major functional modules:

- Expense Entry & Validation (Input Handling)
- Category Management & Display (User Interface)
- History & Reporting (Data Persistence)

4. Non-Functional Requirements: -

These define how the system performs its functions:

- **Performance:** Adding an expense, saving it to disk, and updating the on-screen total must complete instantly, with no noticeable delay.
- **Reliability (Offline by Design):** Since the application never depends on an internet connection, it works identically every time it is launched. If the data file is missing or empty, the system starts with a blank expense list instead of crashing.
- **Usability:** The design has a clean, readable layout (Times New Roman typography, clear labels), and the user can log an expense in under 3 clicks.
- **Error Handling:** The system should handle invalid input-namely, blank, non-numeric, or zero/negative amounts-gracefully, with clear, user-friendly warning messages rather than termination of the system.

5. System Architecture: -

The project follows a Modular Architecture, separating concerns into distinct layers:

- **Presentation Layer (GUI):** Built using Tkinter, this handles all user interactions and displays the expense list and running total.
- **Logic Layer (Validation & Totals):** Contains the pure Python functions for validating input and calculating the running total.
- **Data Layer (Local Storage):** load_expenses() and save_expenses() manage reading and writing all expense records to and from a local expenses.json file.

6. Design Diagrams

6.1 Use Case Diagram: -

- Actor: User

Use Cases:

- Enter Amount
- Select Category
- Add Note (optional)
- Add Expense
- View Expense List
- Delete Selected Expense

- View Running Total

6.2 Workflow Diagram

Start -> Launch App -> Load Saved Expenses from JSON -> User Inputs Amount, Category, Note -> Validate Input -> Save Expense -> Update List & Total -> End.

6.3 Sequence Diagram

1. User enters amount, selects a category, and clicks "Add Expense".

2. GUI reads the amount and note fields.

3. If the amount is blank, non-numeric, or not positive, a warning is shown and the process stops.

4. If valid, a new expense record is created with the date, category, amount, and note.

5. The record is appended to the in-memory expense list.

6. save_expenses() writes the updated list to expenses.json.

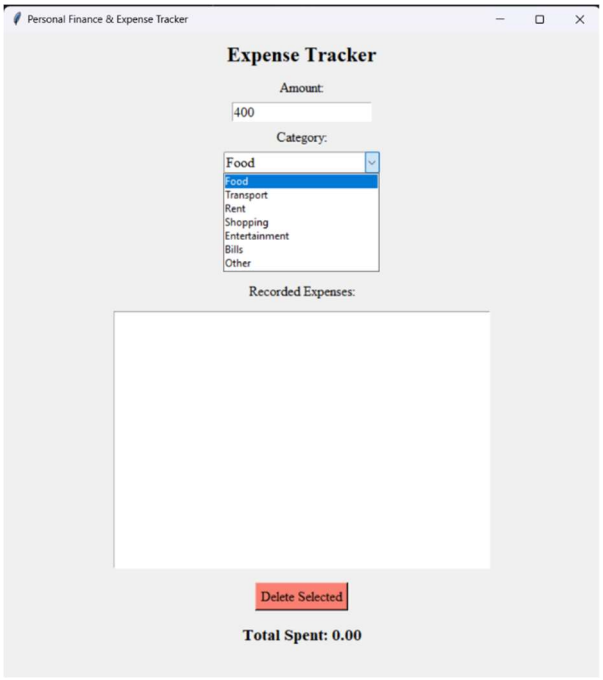
7. GUI calls refresh_list() and update_total() to update the display.

7. Design Decisions & Rationale: -

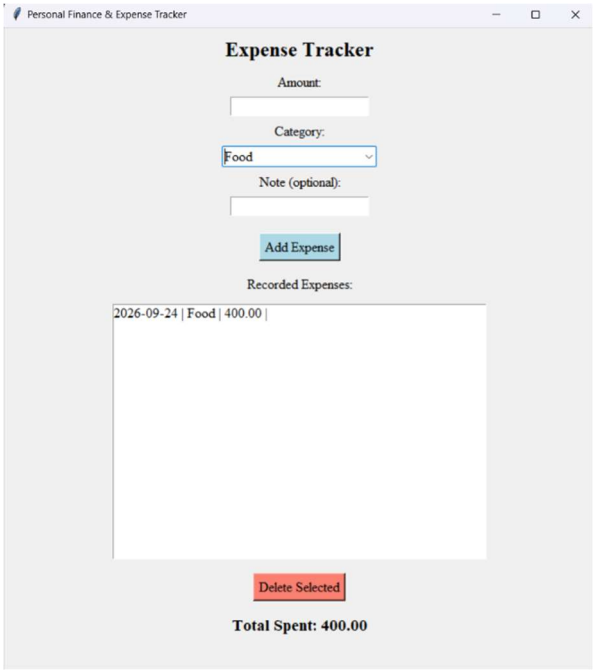
- **Language:** Python was chosen for its simplicity and built-in support for file handling and JSON.
- **GUI Framework:** Tkinter was selected because the requirement was for a lightweight desktop application that runs locally without a browser.
- **Storage:** A local JSON file was selected instead of a database or external API, since expense data is personal and the app needed to work fully offline with simple, human-readable storage.

8. Screenshots: -

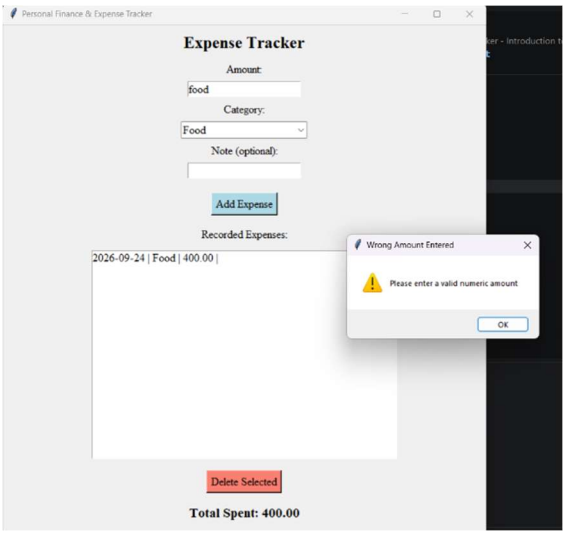
1. Entering the amount, category, and note for a new expense



2.Adding the expense in the record



3. Warning generated when no amount is entered and "Add Expense" is pressed



9. Testing Approach: -

- **Unit Testing:** The total-calculation logic was tested with known values (e.g., three expenses of 100 each should total 300) to ensure accuracy.
- **Boundary Testing:** Tested inputs with 0, negative numbers, blank text, and non-numeric text to ensure the validation logic holds.
- **Persistence Testing:** The application was closed and reopened after adding expenses to verify that all entries reload correctly from expenses.json.

10. Challenges Faced: -

- Ensuring the expense list and total stayed correctly in sync after every add or delete action.
- Handling a missing or empty expenses.json file gracefully on the very first run.
- Designing input validation that rejects blank, non-numeric, and zero/negative amounts without confusing the user.

11. Learnings & Key Takeaways: -

- Gained practical experience in reading and writing JSON data in Python.
- Learned the importance of Modular Design - separating data-handling functions from the GUI made the code easier to follow and debug.
- Understood how to manage Data Persistence using simple local file handling instead of a database.

12. Future Enhancements: -

- Expense Graphs: Add a visualization module (using Matplotlib) to show spending by category.
- Monthly Budget Limits: Allow users to set a monthly budget and warn them when they are close to exceeding it.
- Export to CSV/Excel: Let users export their expense history for use in other tools.

13. References

1. Python Documentation: https://docs.python.org/3/

2. Tkinter Reference: https://tkdocs.com/

3.Python JSON Module Documentation: https://docs.python.org/3/library/json.html